Q1

PART 1 — Create Java Project

1. Open Eclipse IDE
2. Create Project

Click File → New → Java Project

Name: JavaLab

Uncheck: Create module-info.java

Click Finish

3. Create Class

Right click JavaLab

Click New → Class

Name: HelloWorld

Tick:

public static void main

In Package:

com.example

Click Finish

4. Write Code

#1)helloworld

```
package com.example;

public class HelloWorld {
   public static void main(String[] args) {
      System.out.println("Hello World");
   }
}
```

#2)numbers from 1 to 0

```
package com.example;

public class Numbers {
   public static void main(String[] args) {
      for(int i = 1; i <= 10; i++) {
         System.out.println(i);
      }
   }
}
```

#3)Even or Odd

```
package com.example;

import java.util.Scanner;

public class EvenOdd {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a number: ");
      int num = sc.nextInt();

      if(num % 2 == 0) {
         System.out.println("Even");
      } else {
         System.out.println("Odd");
```

```
        }

        sc.close();
    }
}
```

#Grade Calculator:

```
package com.example;

import java.util.Scanner;

public class GradeCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter marks (0–100): ");
        int marks = sc.nextInt();

        if (marks >= 90) {
            System.out.println("Grade: A");
        } else if (marks >= 75) {
            System.out.println("Grade: B");
        } else if (marks >= 60) {
            System.out.println("Grade: C");
        } else if (marks >= 50) {
            System.out.println("Grade: D");
        } else {
            System.out.println("Grade: F");
```

```
        }

        sc.close();
    }
}
```

- 5. Run Program

Click ▶ OR

Right click → Run As → Java Application

PART 2 — Create GitHub Repository

- 6. Open GitHub

Go to https://github.com

Login

- 7. Create Repo

Click + → New repository

Name: java-lab

Click Create repository

- 8. Copy URL

https://github.com/BRR-007/java-lab.git

PART 3 — Connect Eclipse to Git

- 9. Share Project

Right click JavaLab

Click Team → Share Project

Select Git

🔹 10. Important Setting

👉 Tick:

✓ Use or create repository in parent folder of project

🔹 11. Click Finish

PART 4 — Add & Commit

🔹 12. Add files

Right click JavaLab

Click Team → Add to Index

🔹 13. Commit

Right click → Team → Commit

Message:

Initial commit

Click Commit

PART 5 — Push to GitHub

🔹 14. Push

Right click JavaLab

Click Team → Remote → Push

🔹 15. Add Remote

Paste:

https://github.com/BRR-007/java-lab.git

🔹 16. Login

Username: BRR-007

Password: GitHub Token

🔹 17. Push Ref Specification Screen

👉 Click:

All Branches

🔹 18. Next

Click Next

🔹 19. Finish

Click Finish

PART 6 — Verify

🔹 20. Open GitHub

Go to your repo

Refresh page

👉 You will see:

src

HelloWorld.java

JENKINS

9. Create Jenkins job

Step 57

On Jenkins dashboard, click:

New Item

Step 58

Job name:

JavaLabBuild

Step 59

Select:

Freestyle project

Step 60

Click:

OK

10. Configure GitHub repository in Jenkins

Step 61

Scroll to:

Source Code Management

Step 62

Select:

Git

Step 63

In Repository URL, paste:

https://github.com/BRR-007/java-lab.git

Step 64

If repo is public, leave credentials as:

- none -

Step 65

If repo is private, add GitHub username and token as credentials

11. Add build step in Jenkins for Ubuntu

Step 66

Scroll to:

Build

Step 67

Click:

Add Build Step → Execute shell

Step 68

Paste:

```
javac src/com/example/HelloWorld.java
java -cp src com.example.HelloWorld
```

Step 69

Click:

Save

12. Run Jenkins build

Step 70

Click:

Build Now

Step 71

Wait a few seconds

Step 72

Under Builds, click latest build number

#1

or latest number shown

Step 73

Click:

Console Output

Q)Write dowm the steps involved in Sonar installation,installation of sonar scanner CLI,configure SonarQube,set up SonarQube as a service:

SonarCube Installation in Ubuntu:

Step-1:- Install the PostgreSQL if it's not installed on your Ubuntu workstation.

sudo apt install -y postgresql-common postgresql -y

Step-2:- Enable the PostgreSQL database server to automatically start at boot.

sudo systemctl enable postgresql

Step-3:- Start the PostgreSQL database server.

sudo systemctl start postgresql

Step-4:- Log in to the PostgreSQL database server as the postgres user.

sudo -u postgres psql

Step-5:- Create a new sonaruser PostgreSQL role with a strong password to use with SonarQube. Replace your_password with your desired password.

CREATE ROLE sonaruser with login encrypted password:

'your_password';

Step-6:- Create a new sonarqube database.

CREATE DATABASE sonarqube;

Step-7:- Grant the sonaruser role full privileges to the sonarqube database.

GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonaruser;

Step-8:- Switch to the sonarqube database.

\c sonarqube

GRANT ALL PRIVILEGES ON SCHEMA public TO sonaruser;

Step-9:- Exit the PostgreSQL database console.

\q

Install SonarQube:

Step-1:- Update the server's APT package index.

sudo apt update

Step-2:- Install OpenJDK 17.

sudo apt install openjdk-17-jdk -y

Step-3:- Install Unzip to extract files from the SonarQube archive.

sudo apt install unzip

Step-4:- Verify the installed java version.

java -version

sudo wget

https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-25.2.0.102705

.zip

Step-5:- Extract files from the downloaded archive using Unzip.

unzip sonarqube-25.2.0.102705.zip

Step-6:- Move the extracted files to a systemwide directory such as /opt.

sudo mv sonarqube-25.2.0.102705/ /opt/sonarqube

Step-7:- Create a dedicated sonarqube system user without login privileges and a home directory.

sudo adduser --system --no-create-home --group --disabled-login sonarqube

Step-8:- Grant the sonarqube user full privileges to the /opt/sonarqube directory.

sudo chown -R sonarqube:sonarqube /opt/sonarqube

Install SonarScanner CLI:

Step-1:- Visit the SonarScanner CLI page and verify the latest version to download. For example, run the following command to download the SonarScanner CLI version 7.0.1

wget

https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-scanner-cli-linux-x64.zip

Step-2:- Extract files from the archive depending on the downloaded version.

unzip sonar-scanner-cli-linux-x64.zip

Step-3:- Move the extracted directory to /opt/sonarscanner.

sudo mv sonar-scanner-linux-x64/ /opt/sonarscanner

Step-4:- Open the sonar-scanner.properties configuration file.

sudo nano /opt/sonarscanner/conf/sonar-scanner.properties

Step-5:- Find the following sonar.host.url directive and change the default https://mycompany.com/sonarqube value to 127.0.0.1.

sonar.host.url=127.0.0.1

sudo chmod +x /opt/sonarscanner/bin/sonar-scanner

Step-6:- Link the sonar-scanner binary to the /usr/local/bin directory to enable it as a system-wide command.

sudo ln -s /opt/sonarscanner/bin/sonar-scanner /usr/local/bin/sonar-scanner

Step-7:- View the installed SonarScanner version.

sonar-scanner -v

Configure SonarQube:

Step-1:- Open the main sonar.properties Sonarqube configuration file.

sudo nano /opt/sonarqube/conf/sonar.properties

Step-2:- Add the following configurations at the end of the file. Replace sonaruser and your_password with actual PostgreSQL database user details.

sonar.jdbc.username=sonaruser

sonar.jdbc.password=your_password

sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube

sonar.web.javaAdditionalOpts=-server

sonar.web.host=0.0.0.0

sonar.web.port=9000

sudo nano /etc/sysctl.conf

Step-3:- Add the following directives at the end of the file.

vm.max_map_count=524288

fs.file-max=131072

sudo nano /etc/security/limits.d/99-sonarqube.conf

Step-4:- Add the following directives to increase the file descriptor and process limits for

SonarQube.

sonarqube - nofile 131072

sonarqube - nproc 8192

sudo ufw allow 9000/tcp

● Run the following command to install UFW and allow SSH connections if it's unavailable.

sudo apt install ufw -y && sudo ufw allow 22/tcp

Step-5:- Reload UFW to apply the firewall configurations.

sudo ufw reload

Step-6:- View the UFW status and verify that below are the only active firewall rules.

sudo ufw status

Set Up SonarQube as a System Service:

Step-1:- Create a new sonarqube.service file.

sudo nano /etc/systemd/system/sonarqube.service

Step-2:- Add the following configurations to the file.

```
[Unit]
Description=SonarQube service
After=syslog.target network.target
[Service]
Type=forking
ExecStart=/opt/sonarqube/bin/linux-x86-64/sonar.sh start
ExecStop=/opt/sonarqube/bin/linux-x86-64/sonar.sh stop
User=sonarqube
Group=sonarqube
PermissionsStartOnly=true
Restart=always
StandardOutput=syslog
LimitNOFILE=131072
LimitNPROC=8192
TimeoutStartSec=5
SuccessExitStatus=143
[Install]
WantedBy=multi-user.target
```

sudo systemctl daemon-reload

Step-3:- Enable SonarQube to start at boot.

sudo systemctl enable sonarqube

Step-4:-Start the SonarQube service.

sudo systemctl start sonarqube

Step-5:- View the SonarQube service status and verify that it's running.

sudo systemctl status sonarqube

sudo reboot now

Q4).write down the steps involved in installing docker on windows and write the docker commands for the following:

STEPS:

Installing Docker on windows

1.open https://docs.docker.com

2.Download the latest version and install it.

3.To execute docker commands in windows

-->open powershell

-->run the docker command

COMMANDS:

1. To see the list of docker images in our machine

>>docker images

(or)

>> docker image ls

2. To download an image from docker hub

>> docker pull image-name

3.To upload an image into dockerhub

>> docker push image-name

4. To search for an images on docker hub

>> docker search image-name

5. To delete an image on docker host

>> docker rmi image-name

6. To create an image from a container

>> docker commit container-name/container-id image-name

7. To create an image from a dockerfile

>> docker build -t imagename .

8.To see the list of running containers

>> docker container ls

9.To see the list of stopped and running containers

>> docker ps -a

10. To start a container

>> docker start container-name/container-id

11. To stop a container

>> docker stop container-name/container-id

12. To restart a container

>> docker restart container-name/container-id

To restart a container after 10 seconds

>> docker restart -t 10 container-name/container-id

13. To remove a stopped container

>> docker rm container-name/container-id

14. To remove a running container

>> docker rm -f container-name/container-id

15. To stop all the running containers

>>docker stop $(docker ps -aq)

16. To see the logs generated by the container 16.

>> docker logs container-name/container-id

17. To see the ports of a container 17.

>> docker port container-name/container-id

18. To find detailed info about a container 18.

>> docker inspect container-name/container-id

19.creating a docker container 19.

>> docker run image-name

20.To inspect a network 20

>> docker network inspect network-id/network-name

2 1. Remove all stopped containers

>>docker rm $(docker ps -aq)

22. Remove all running containers

>>docker rm $(docker ps -aq)

2 3. List docker networks

>>docker network ls

2 4. Create a network

>>docker network create network-name

2 5. Delete a network

>>docker network rm network-id/network-name

1. Create Java Project in IntelliJ IDEA

Step 1

Open IntelliJ → Click **New Project**

Step 2

- Select **Java**
- Enter project name:

GradeCalculator

Step 3

✓ Tick:

Create Git repository

Step 4

Click **Create**

---

2. Create Java Class

Step 5

Right click **src**

Step 6

Click:

New → Java Class

Step 7

Enter name:

GradeCalculator

---

3. Write Program

Step 8

Type the code:

```
import java.util.Scanner;

public class GradeCalculator {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);
      int marks = sc.nextInt();

      if (marks >= 90) {
         System.out.println("Grade: A");
      } else if (marks >= 75) {
```

```
        System.out.println("Grade: B");
    } else if (marks >= 60) {
        System.out.println("Grade: C");
    } else if (marks >= 50) {
        System.out.println("Grade: D");
    } else {
        System.out.println("Grade: F");
    }

    sc.close();
  }
}
```

---

4. Run Program

Step 9

Click ▶ Run

Step 10

Enter input:

85

Step 11

Output:

Grade: B

---

5. Create Repository in GitHub

Step 12

Open GitHub → Login

Step 13

Click:

New repository

Step 14

Repository name:

grade-calculator

Step 15

Click:

Create repository

---

6. Commit Project in IntelliJ

Step 16

Click **Commit (left side icon)**

Step 17

Select file:
✓ GradeCalculator.java

Step 18

Enter message:

initial commit

Step 19

Click:

Commit

---

7. Add Remote Repository

Step 20

Click:

Git → Manage Remotes

Step 21

Click **+**

Step 22

Enter:

- Name:

origin

- URL:

https://github.com/BRR-007/grade-calculator.git

Step 23

Click **OK**

---

8. Push to GitHub

Step 24

Click:

Git → Push

Step 25

Login using:

- Username: BRR-007
- Password: **GitHub Token**

Step 26

Click:

Push

---

9. Verify

Step 27

Open GitHub repository

You should see:

GradeCalculator.java